

ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia a informatiky

Semestrálna práca 2 Diskrétna simulácia

Michael Holý

Udalostná simulácia – Stolárska dielňa

Cvičiaci : Ing. Peter Jankovič, Phd.

Žilina, 2025

Obsah

1	Zadanie semestrálnej práce	4
2	Popis implementácie generátorov pre jednotlivé rozdelenia	5
2.1	Trieda ContinuousExponentialGenerator	5
2.2	Trieda ContinuousTriangularGenerator	5
2.3	Vykonané štatistické testy implementovaných generátorov	6
2.3.1	Generátor exponenciálneho rozdelenia	6
2.3.2	Generátor trojuholníkového rozdelenia	6
3	Popis implementácie všeobecného udalostného simulačného jadra a ostatných všeobecných tried	7
3.1	Trieda GeneralMonteCarloSimulation	7
3.2	Trieda GeneralEventDrivenSimulation	8
3.3	Trieda GeneralEvent	9
3.4	Trieda SystemEvent	9
3.5	Rozhranie ISimDelegate	9
3.6	Trieda Statistics	10
3.7	Trieda WeightedStatistics	10
4	Udalostný diagram a jeho popis	11
4.1	Popis obsahu udalostného diagramu	12
4.2	Popis vzťahov a skratiek v udalostnom diagrame	12
5	Implementácia simulácie stolárskej dielne a jednotlivých udalostí	13
5.1	Popis implementácie jednotlivých entít v simulácii	13
5.1.1	Trieda Workplace	13
5.1.2	Trieda Worker	13
5.1.3	Trieda Order	14
5.2	Trieda FurnitureWorkshopSimulation	14
5.3	Trieda OrderArrivalEvent	16
5.4	Trieda FinishMaterialPrepEvent	16
5.5	Trieda MoveBetweenWarehouseWorkplaceEvent	17
5.6	Trieda MoveBetweenWorkplacesEvent	17
5.7	Trieda FinishCuttingEvent	17
5.8	Trieda FinishVarnishingEvent	17
5.9	Trieda FinishAssemblyEvent	18
5.10	Trieda FinishForgingEvent	18
5.11	Trieda FinalizationOfOrderEvent	18
6	Vykonané experimenty	19
6.1	Úvodné experimenty	19

6.2	Následné experimenty zamerané na skupinu C.....	19
6.3	Následné experimenty zamerané na skupinu A a B.....	20
6.4	Následné experimenty opäť zamerané na skupinu C	21
6.5	Vyčíslenie štatistík pre optimálne riešenie (10 000 replikácií).....	22
7	Záver	23

1 Zadanie semestrálnej práce

Cieľom semestrálnej práce je vytvoriť udalostne orientovaný simulačný model pre stolársku dielňu Najlepší nábytok, s.r.o., ktorá vyrába nábytok na zákazku. Cieľom simulácie je zistiť minimálny počet stolárov v troch špecializovaných skupinách (A, B, C), ktorý zabezpečí, že priemerný čas od prijatia objednávky po jej dokončenie nepresiahne 16 hodín.

Úlohy:

- Implementovať flexibilný generátor exponenciálneho a trojuholníkového rozdelenia.
- Navrhnuť a implementovať všeobecné udalostné simulačné jadro.
- Vytvoriť udalostný diagram pre implementovanú simuláciu na základe zadania semestrálnej práce.
- Implementovať udalostnú simuláciu pre stolársku dielňu na základe udalostného diagramu, vrátane spôsobu výpočtu štatistík pre sledované veličiny.
- Vytvorenie GUI umožňujúce spustiť simuláciu v režime s maximálnou rýchlosťou a vykresľovaním ustálovania priemerného času od prijatia objednávky po jej dokončenie do grafu, v režime s možnosťou spomalenia a zrýchlenia simulácie ako aj podrobnejším zobrazením jednotlivých stavov sledovaných veličín v systéme a možnosťou zobrazenia priebežných ale aj globálnych výsledkov počítaných štatistík.
- Dodržať princípy oddelenia užívateľského rozhrania od jadra aplikácie.
- Vykonanie experimentov pre určenie optimálneho počtu pracovníkov jednotlivých skupín.

2 Popis implementácie generátorov pre jednotlivé rozdelenia

V tejto časti sa venujem popisu implementácie generátorov náhodných čísiel, ktoré bolo potrebné implementovať v rámci tejto semestrálnej práce. Ostatné použité generátory boli implementované v predošlej semestrálnej práci.

2.1 Trieda ContinuousExponentialGenerator

Trieda **ContinuousExponentialGenerator** predstavuje generátor spojitého exponenciálneho rozdelenia.

Konštruktor prijíma strednú hodnotu intervalu, v ktorom sa majú generovať čísla, pričom generátor náhodných čísiel v tejto triede je inicializovaný pomocou seedu vygenerovaným generátorom seedov.

Metóda nextDouble() vygeneruje a vráti náhodné desatinné číslo podľa exponenciálneho rozdelenia, pričom používa inverznú transformáciu $X = -\mu * \ln(1-U)$.

2.2 Trieda ContinuousTriangularGenerator

Trieda **ContinuousTriangularGenerator** predstavuje generátor náhodných čísiel zo spojitého trojuholníkového rozdelenia.

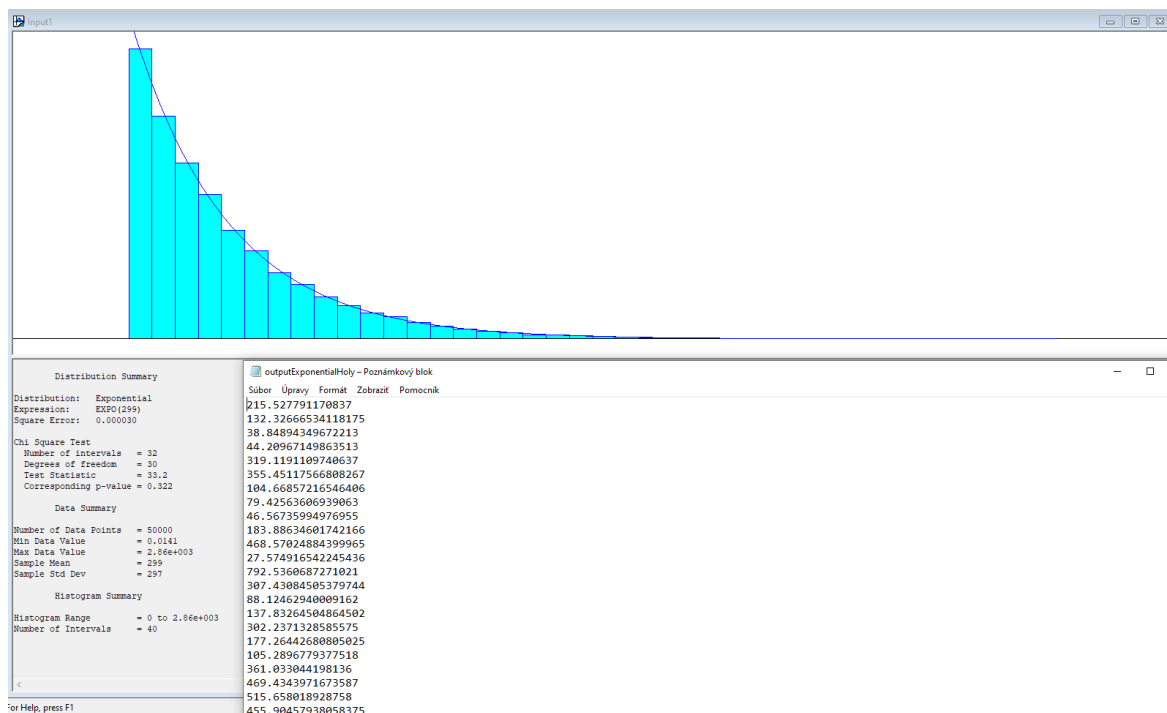
Konštruktor prijíma minimálnu hodnotu, maximálnu hodnotu a vrchol rozdelenia pre interval, v ktorom má generovať čísla, pričom generátor náhodných čísiel v tejto triede je inicializovaný pomocou seedu vygenerovaným generátorom seedov.

Metóda nextDouble() vygeneruje a vráti desatinné číslo zo zvoleného trojuholníkového intervalu na základe štandardného vzorca pre výpočet náhodného čísla z trojuholníkového rozdelenia.

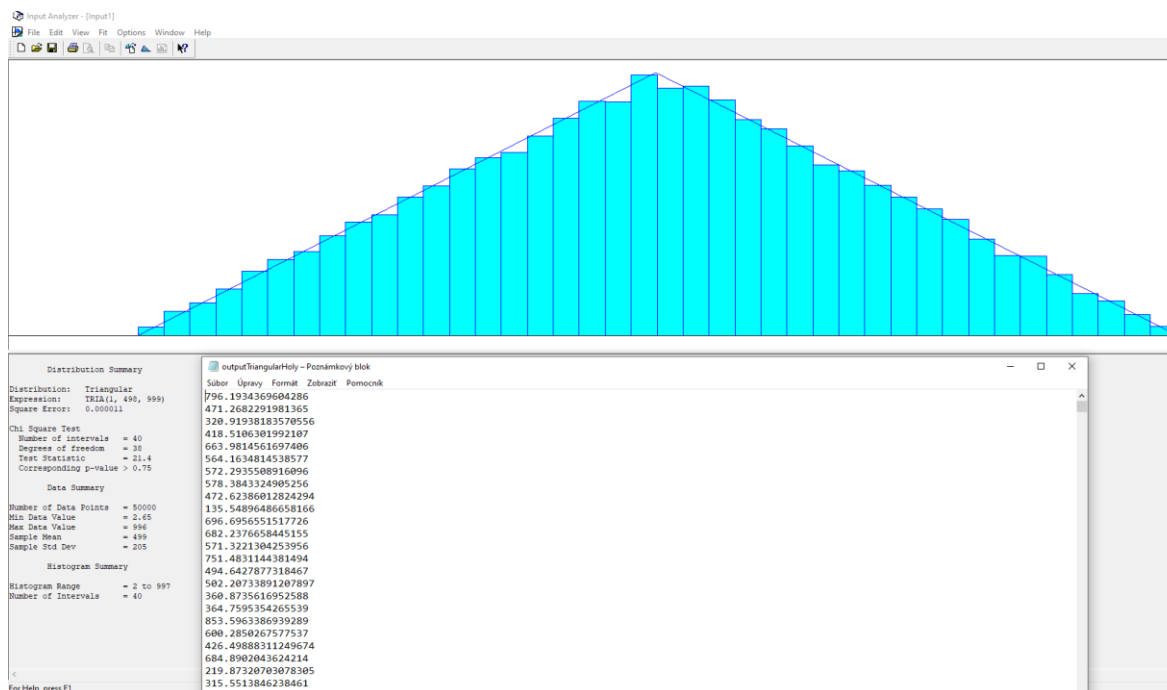
2.3 Vykonané štatistické testy implementovaných generátorov

Správnosť implementovaných generátorov sme overili pomocou **Chi-Square štatistických testov**.

2.3.1 Generátor exponenciálneho rozdelenia



2.3.2 Generátor trojuholníkového rozdelenia



3 Popis implementácie všeobecného udalostného simulačného jadra a ostatných všeobecných tried

V tejto časti sa venujem popisu implementácie všeobecného udalostného jadra, ako aj popisu implementácie všeobecných tried využívaných pre riadenie a spracovanie simulácie stolárskej dielne.

3.1 Trieda **GeneralMonteCarloSimulation**

Trieda **GeneralMonteCarloSimulation** predstavuje všeobecné simulačné jadro pre simulácie metódou Monte Carlo, ktoré sme implementovali v predchádzajúcej semestrálnej práci. Definuje štruktúru na spúšťanie opakovaných simulácií (replikácií), pričom umožňuje ich zastavenie pomocou metódy **stopSimulation()**.

Všetky podtriedy musia implementovať tieto metódy aby definovali svoju vlastnú simuláciu Monte Carlo:

- **beforeReplication()** – časť simulácie vykonaná pred samotnou replikáciou
- **virtualReplication()** – hlavná časť simulácie
- **afterReplication()** – časť simulácie vykonaná po replikácii (napr. na spracovanie výsledkov).

Taktiež táto trieda umožňuje podtriedam prekryť metódy **beforeAllReplications()** a **afterAllReplications()** v prípade, že potrebujú implementovať časti simulácie, ktoré sa vykonajú pred a po všetkých vykonaných replikáciách.

3.2 Trieda GeneralEventDrivenSimulation

Trieda **GeneralEventDrivenSimulation** predstavuje všeobecné simulačné jadro pre udalostne orientované simulácie, ktoré je postavené na základe všeobecného Monte Carlo jadra (**GeneralMonteCarloSimulation**). Táto trieda definuje základ pre simulácie, kde sa systém vyvíja diskkrétne skokmi medzi jednotlivými udalosťami v čase, usporiadanými vo pomocou **prioritného frontu**..

Trieda uchováva **aktuálny simulačný čas** a **maximálny čas jednej replikácie**, všetky naplánované udalosti sú uložené v **prioritnom fronte – kalendári udalostí** zoradené podľa času vykonania.

Trieda podporuje **pozastavenie** a **spomalenie simulácie**, ako aj podporu pre komunikáciu s GUI prostredníctvom **návrhového vzoru Observer**.

Hlavné metódy:

- **beforeReplication()** - Inicializuje simuláciu pred každou replikáciou. Vyčistí front udalostí, resetuje čas a aktivuje prípadné spomalenie simulácie.
- **virtualReplication()** – Hlavná simulačná časť, cyklus while ktorý postupne vyberá a vykonáva jednotlivé udalosti z frontu. Simulácia sa zastaví, ak je event za hranicou maximálneho času alebo ak je vyvolané prerušenie.
- **afterReplication()** – Metóda volaná po každej replikácii, ktorú možno v podtriedach ďalej rozšíriť.
- **addEvent()** – Pridá novú udalosť do frontu. Udalosti nesmú byť naplánované na čas v minulosti (kontrola kauzality).
- **pauseSimulation()** / **resumeSimulation()** – Pozastaví alebo obnoví simuláciu.
- **turnOnSlowdown()** – Aktivuje spomalenie simulácie pomocou naplánovania špeciálnej systémovej udalosti, ktorá sa následne plánuje sama v pravidelných intervaloch.
- **registerDelegate()** – Umožňuje zaregistrovať pozorovateľov, ktorým sa zasielajú notifikácie o zmene stavu simulácie.
- **refreshGUI()** – Umožňuje simulácii informovať GUI o zmene v simulácii.

3.3 Trieda GeneralEvent

Trieda GeneralEvent je abstraktná trieda pre všetky udalosti v rámci udalostne orientovanej simulácie. Táto trieda definuje a uchováva **čas vykonania udalosti** a **referenciu na simuláciu**, v ktorej sa udalosť vykonáva.

Metódy:

- **execute()** – abstraktná metóda, ktorú musí implementovať každá podtrieda. Definuje konkrétne správanie udalosti pri jej vykonaní.
- **compareTo()** – umožňuje triedenie udalostí v **prioritnom fronte** podľa času vykonania (od najskoršej po najneskoršiu).
- **getTime()** – vracia naplánovaný čas vykonania udalosti.

3.4 Trieda SystemEvent

Trieda SystemEvent je špeciálny typ udalosti slúžiaci na **riadenie spomalenia simulácie** v prípade, že je aktivovaný režim spomalenia. Je odvodená z triedy GeneralEvent a automaticky sa opakovane plánuje.

Metóda execute():

- Skontroluje, či je aktívne spomalenie a či je zadaný nenulový pomer spomalenia.
- Uspí vlákno na vypočítaný čas podľa vzorca $\text{sleepTime} = 1000 / \text{slowdownRatio}$.
- Naplánuje ďalšiu systémovú udalosť na $\text{currentSimulationTime} + 1$.
- Vyvolá aktualizáciu GUI cez `refreshGUI(RefreshType.REFRESH_TIME)`.

3.5 Rozhranie ISimDelegate

Rozhranie ISimDelegate definuje rozhranie pre komponenty reagujúce na priebeh simulácie pomocou metódy **Refresh()**.

Definuje aj **Enum RefreshType**, ktorý slúži na rozlíšenie typu zmeny, ktorá má byť spracovaná.

3.6 Trieda Statistics

Trieda Statistics slúži na **zber a štatistické vyhodnotenie údajov** z replikácií simulácie. Umožňuje priebežné zaznamenávanie hodnôt a následne umožňuje aj výpočet **priemeru, výberovej smerodajnej odchýlky a intervalu spoľahlivosti** pre zadané úrovne (95 % a 99 %).

Metódy:

- **addData()** – pridanie hodnôt do štatistiky.
- **getMean()** – výpočet aritmetického priemeru.
- **getSampleStandardDeviation()** – výpočet výberovej smerodajnej odchýlky.
- **getConfidenceInterval()** – výpočet intervalu spoľahlivosti (95 % alebo 99 %) podľa veľkosti vzorky s využitím t-tabuliek pre malé n a Z hodnoty pre veľké n.

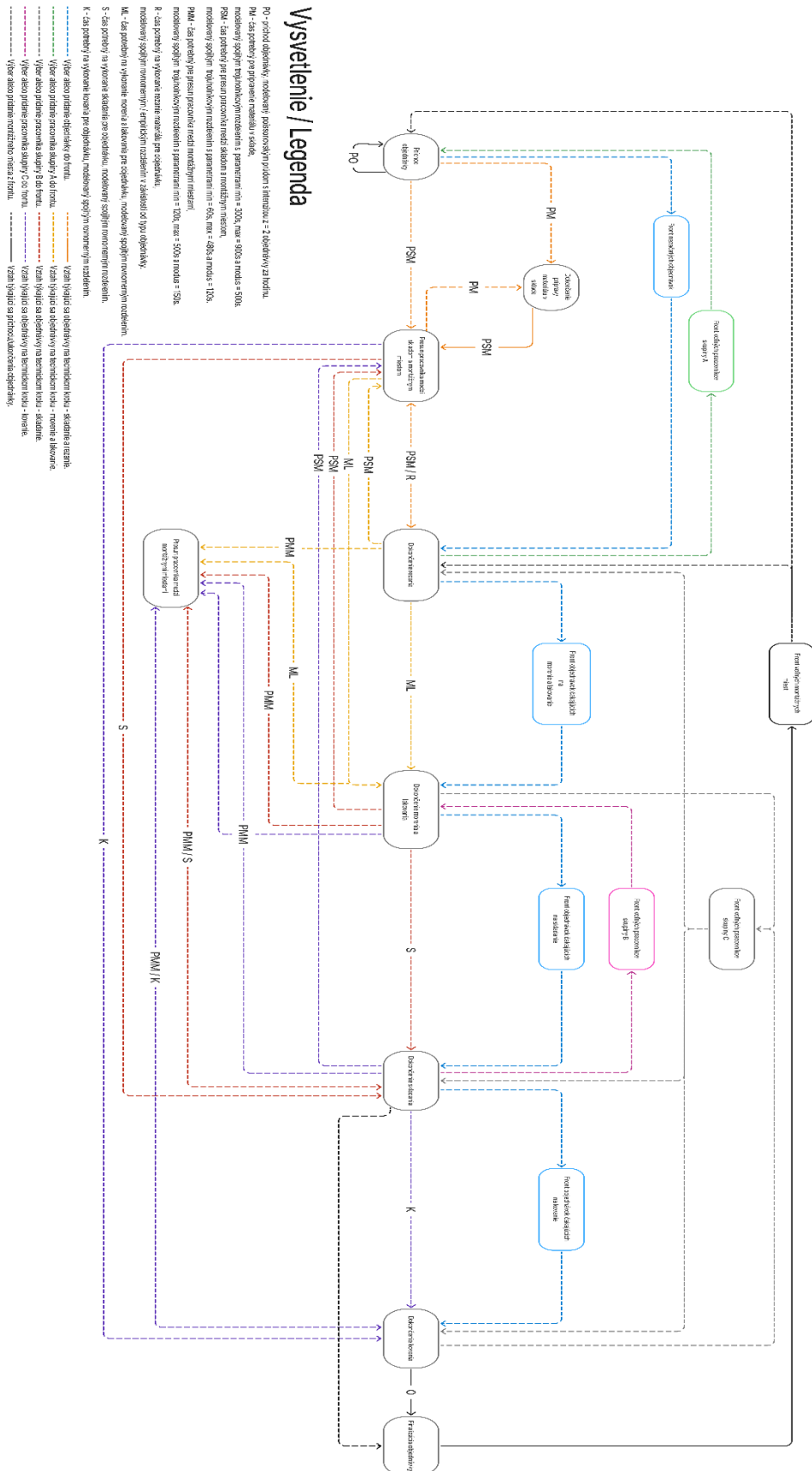
3.7 Trieda WeightedStatistics

Trieda WeightedStatistics slúži na **zber a vyhodnocovanie vážených štatistických údajov**. Umožňuje priebežné zaznamenávanie hodnôt a následne umožňuje aj výpočet **váženého priemeru, váženej výberovej smerodajnej odchýlky a intervalu spoľahlivosti** pre zadané úrovne (95 % a 99 %).

Metódy:

- **addData()** – pridanie hodnôt s váhou do štatistiky.
- **getMean()** – výpočet váženého aritmetického priemeru.
- **getWeightedSampleStandardDeviation()** – výpočet váženej výberovej smerodajnej odchýlky.
- **getConfidenceInterval()** – výpočet váženého intervalu spoľahlivosti pomocou t-tabuliek (pre malé n) alebo Z hodnoty (pre veľké n).

4 Udalostný diagram a jeho popis



4.1 Popis obsahu udalostného diagramu

Udalostný diagram znázorňuje logiku podľa ktorej som implementoval riešenie mojej semestrálnej práce. Zobrazuje kľúčové prvky systému a ich vzájomné vzťahy, ktoré modelujú priebeh prijatia objednávky až po jej dokončenie.

Diagram pozostáva z:

- **Viacerych typov frontov** – napr. front nezačatých objednávok, fronty čakajúcich pracovníkov alebo objednávok v rôznych fázach výroby,
- **Udalostí** – reprezentujúce prijatie a dokončenie objednávky alebo priebeh jednotlivých krokov potrebných pre dokončenie objednávky ako rezanie, morenie a lakovanie, skladanie, montáž kovaní a presun pracovníkov z/do skladu a montážnych miest.
- **Väzieb medzi udalosťami a frontami** - ktoré sú znázornené pomocou **plných a prerušovaných čiar** (plná čiara = udalosť sa naplánuje vždy, prerušovaná čiara = udalosť sa naplánuje po splnení určitých podmienok) s rôznym farebným rozlíšením (zahrnutý v legende).
- **Skratiek** označujúcich časové trvanie pre vykonanie danej udalosti, ktorých význam je uvedený v legende.

4.2 Popis vzťahov a skratiek v udalostnom diagrame

Vysvetlenie / Legenda

PO - príchod objednávky, modelovaný poissonovským prúdom s intenzitou $\lambda = 2$ objednávky za hodinu.

PM - čas potrebný pre prípravu materiálu v sklade,

modelovaný spojitým trojuholníkovým rozdelením s parametrami $\min = 300s$, $\max = 900s$ a $\text{modus} = 500s$.

PSM - čas potrebný pre presun pracovníka medzi skladom a montážnym miestom,

modelovaný spojitým trojuholníkovým rozdelením s parametrami $\min = 60s$, $\max = 480s$ a $\text{modus} = 120s$.

PMM - čas potrebný pre presun pracovníka medzi montážnymi miestami,

modelovaný spojitým trojuholníkovým rozdelením s parametrami $\min = 120s$, $\max = 500s$ a $\text{modus} = 150s$.

R - čas potrebný na vykonanie rezanie materiálu pre objednávku,

modelovaný spojitým rovnomerným / empirickým rozdelením v závislosti od typu objednávky.

ML - čas potrebný na vykonanie morenia a lakovania pre objednávku, modelovaný spojitým rovnomerným rozdelením.

S - čas potrebný na vykonanie skladania pre objednávku, modelovaný spojitým rovnomerným rozdelením.

K - čas potrebný na vykonanie kovania pre objednávku, modelovaný spojitým rovnomerným rozdelením.

----- Výber alebo pridanie objednávky do frontu.

----- Výber alebo pridanie pracovníka skupiny A do frontu.

----- Výber alebo pridanie pracovníka skupiny B do frontu.

----- Výber alebo pridanie pracovníka skupiny C do frontu.

----- Výber alebo pridanie montážneho miesta z frontu.

----- Vzťah týkajúci sa objednávky na technickom kroku - skladanie a rezanie.

----- Vzťah týkajúci sa objednávky na technickom kroku - morenie a lakovanie.

----- Vzťah týkajúci sa objednávky na technickom kroku - skladanie.

----- Vzťah týkajúci sa objednávky na technickom kroku - kovanie.

----- Vzťah týkajúci sa príchodu/ukončenia objednávky.

5 Implementácia simulácie stolárskej dielne a jednotlivých udalostí

V tejto časti sa venujem popisu implementácie simulácie stolárskej dielne ako aj popisu implementácie jednotlivých entít a udalostí, ktoré figurujú v tejto simulácii.

5.1 Popis implementácie jednotlivých entít v simulácii

5.1.1 Trieda Workplace

Trieda Workplace reprezentuje **jedno montážne miesto** v stolárskej dielni, na ktorom sa vykonávajú jednotlivé technologické operácie počas výroby nábytku. Každé pracovisko má **identifikačné číslo** a **stav**, ktorý popisuje, čo sa na ňom práve deje (Voľné, Čakajúce na pracovníka, Prebieha rezanie, atď...).

Táto trieda umožňuje získavať a nastavovať potrebné informácie o pracovisku pomocou **Get** a **Set** metód a taktiež implementuje **rozhranie Comparable**, aby ho bolo možné vkladať do prioritného frontu (výber pracoviska s najnižším číslom).

5.1.2 Trieda Worker

Trieda Worker reprezentuje **pracovníka** v simulácii stolárskej dielne. Každý pracovník patrí do jednej z troch skupín (A, B alebo C) a má vlastné ID, polohu, stav (Voľný, Presúva sa, Reže, atď...) a informácie o svojej zaneprázdnenosti. Pomocou atribútov **isBusy**, **totalFreeTime** a **lastFreeStartTime** sa vypočítava **percentuálne vytťaženie pracovníka** pre neskoršie vyhodnotenie štatistík.

Hlavné metódy:

- **setBusy()** – Aktualizuje stav zaneprázdnenosti a zaznamenáva časy voľna.
- **getWorkloadPercentage()** – Vypočíta percentuálne vytťaženie pracovníka na základe dĺžky simulácie.
- **setState()** – Nastavuje aktuálny stav pracovníka.

5.1.3 Trieda Order

Trieda Order reprezentuje **prijatú objednávku** na výrobu konkrétneho nábytku. Každá objednávka má priradený typ nábytku (Stôl, Stolička, Skriňa), pracovisko, pracovníka a sleduje svoj aktuálny stav (Nezačatá, V procese vyhotovenia, Dokončená) počas výrobného procesu a taktiež zaznamenáva aj poslednú dokončenú časť objednávky (Rezanie, Morenie a lakovanie, atď...). Pomocou atribútu `SimulationTimeAtStart` sa zaznamenáva čas, kedy objednávka prišla do systému a neskôr je využitá pri výpočte času, koľko trvalo objednávku dokončiť.

Táto trieda umožňuje získavať a nastavovať potrebné informácie o objednávke pomocou **Get** a **Set** metód.

5.2 Trieda FurnitureWorkshopSimulation

Trieda FurnitureWorkshopSimulation predstavuje konkrétnu implementáciu **udalostne orientovanej simulácie** výrobného procesu v stolárskej dielni. Rozširuje základné simulačné jadro **GeneralEventDrivenSimulation** a definuje všetky potrebné entity, fronty, štatistiky a generátory potrebné na beh simulácie ako aj jej následné spracovanie v GUI. Je dôležité zdôrazniť, že simulačný čas bol **počítaný v sekundách** a taktiež je dôležité zdôrazniť, že všetky fronty v simulácii sú **implementované ako fronty FIFO** (First in – First out) okrem frontu pre voľné montážne miesta (Prioritný front), na základe čoho sa vyberajú aj všetky voľné entity v priebehu simulácie (čiže všetci voľní pracovníci alebo objednávky vo frontoch sú vyberané štýlom FIFO).

Trieda je zodpovedná za:

- Inicializáciu pracovníkov, pracovísk a generátorov využívaných v simulácii.
- Správu FIFO frontov čakajúcich objednávok v jednotlivých častiach výroby a FIFO frontov voľných pracovníkov skupín A, B a C.
- Správa štruktúr pre uchovávanie aktívnych entít potrebných pre zobrazenie v GUI počas behu replikácie.
- Správu a vyhodnocovanie sledovaných štatistík

Sledované štatistiky:

- Priemerný čas dokončenia objednávky.
- Priemerný počet nezačatých objednávok.
- % pracovné zaťaženie jednotlivých stolárov zo skupiny A,B a C.
- % pracovné zaťaženie pre celú skupinu A, B a C.

Nastavenia jednotlivých generátorov:

Náhodný jav	Rozdelenie	Nastavenie
Prúd objednávok	Exponenciálne	E = 1800
Typ objednávky	Diskrétné empirické	U = <1;2); p = 0.5 U = <2;3); p = 0.15 U = <3;4); p = 0.35
Presun sklad-m.miesto	Spojité trojuholníkové	Min = 60; Max = 480; Modus = 120
Presun medzi m.miest.	Spojité trojuholníkové	Min = 120; Max = 500; Modus = 150
Príprava materiálu	Spojité trojuholníkové	Min = 300; Max = 900; Modus = 500

Typ nábytku	Technologický krok	Rozdelenie	Nastavenie
Stôl	Rezanie	Spojité empirické	U = <600;1500); p = 0.6 U = <1500;3000); p = 0.4
	Morenie a lakovanie	Spojité rovnomerné	U = <12000;36600)
	Skladanie	Spojité rovnomerné	U = <1800;3600)
Stolička	Rezanie	Spojité rovnomerné	U = <720;960)
	Morenie a lakovanie	Spojité rovnomerné	U = <12600;32400)
	Skladanie	Spojité rovnomerné	U = <840;1440)
Skriňa	Rezanie	Spojité rovnomerné	U = <900;4800)
	Morenie a lakovanie	Spojité rovnomerné	U = <36000;42000)
	Skladanie	Spojité rovnomerné	U = <2100;4500)
	Montáž kovaní	Spojité rovnomerné	U = <900;1500)

Metódy:

- **beforeAllReplications()** – Resetuje globálne štatistiky pred spustením série replikácií.

- **beforeReplication()** – Inicializuje stav simulácie pred každou replikáciou, naplánuje prvú udalosť a resetuje štatistiky pre replikáciu.
- **afterReplication()** –Aktualizuje globálne hodnoty (pokiaľ nebola simulácia predčasne pzastavená).
- **afterAllReplications()** – Po ukončení všetkých replikácií aktualizuje GUI podľa režimu simulácie.
- **getAverageWorkloadPercentageForWholeGroup()** – Vráti priemerné vyťaženie skupiny A, B alebo C.
- **updateWorkloadStatisticsForAllWorkers()** – Uloží vyťaženosť jednotlivých pracovníkov do globálnych štatistík.
- **resetAllGlobalStatistics()** – Resetuje štatistiky naprieč všetkými replikáciami.
- **resetAllReplicationStatistics()** – Resetuje štatistiky pre aktuálnu replikáciu.
- **resetAllQueues()** – Resetuje všetky fronty pracovísk, objednávok a zákazníkov, zákazníkov znovu inicializuje a priradí do potrebných frontov / štruktúr pre ich sledovanie.

5.3 Trieda OrderArrivalEvent

Trieda OrderArrivalEvent predstavuje **príchod novej objednávky**. Vytvorí objednávku s náhodne vygenerovaným typom a priradí ju voľnému pracovníkovi skupiny A a montážnemu miestu (ak sú dostupní), inak ju zaradí do frontu nezačatých objednávok. Ak je pracovník v sklade, rovno sa naplánuje udalosť **FinishMaterialPrepEvent**, inak sa najprv naplánuje **MoveBetweenWarehouseWorkplaceEvent**. Zároveň aktualizuje štatistiky nezačatých objednávok a naplánuje príchod ďalšej objednávky. Nakoniec sa aktualizuje GUI.

5.4 Trieda FinishMaterialPrepEvent

Trieda FinishMaterialPrepEvent reprezentuje udalosť dokončenia prípravy materiálu pracovníkom. Zmení dokončenú fázu objednávky a stav pracovníka, pričom naplánuje udalosť **MoveBetweenWarehouseWorkplaceEvent** pre jeho presun na montážne miesto. Na záver aktualizuje GUI.

5.5 Trieda **MoveBetweenWarehouseWorkplaceEvent**

Trieda `MoveBetweenWarehouseWorkplaceEvent` reprezentuje udalosť presunu pracovníka zo skladu k montážnemu miestu alebo opačne a v závislosti od poslednej dokončenej fázy objednávky spúšťa nasledujúcu činnosť (napr. príprava materiálu, rezanie, lakovanie, skladanie alebo montáž kovaní). Nastavuje stav pracovníka a montážneho miesta, naplánuje príslušnú udalosť dokončenia nasledujúcej **výrobnej fázy objednávky** a aktualizuje GUI.

5.6 Trieda **MoveBetweenWorkplacesEvent**

Trieda `MoveBetweenWorkplacesEvent` reprezentuje udalosť presunu pracovníka medzi montážnymi miestami. Nastaví stav pracovníka a pracoviska a na základe poslednej dokončenej fázy (rezanie, lakovanie alebo skladanie) naplánuje udalosť dokončenia nasledujúcej výrobnnej fázy a aktualizuje GUI.

5.7 Trieda **FinishCuttingEvent**

Trieda `FinishCuttingEvent` reprezentuje udalosť ukončenia rezania objednávky pracovníkom A. Ak sú v systéme ďalšie nezačaté objednávky, A sa priradí k ďalšej a naplánuje sa udalosť **`MoveBetweenWarehouseWorkplaceEvent`**. Ak nie, stáva sa voľným. Následne sa pokúsi priradiť voľný pracovník C prioritne na existujúcu objednávku čakajúcu na montáž kovaní (naplánovanie udalosti **`FinishForgingEvent`**, alebo príslušnej **`Move..Event`** na základe pozície voľného pracovníka), alebo priamo na lakovanie aktuálne dokončenej objednávky (naplánovanie udalosti **`FinishForgingEvent`**, alebo príslušnej **`Move..Event`** na základe pozície voľného pracovníka). Ak C nie je voľný, objednávka sa zaradí do frontu na lakovanie. Nakoniec sa aktualizuje GUI.

5.8 Trieda **FinishVarnishingEvent**

Trieda `FinishVarnishingEvent` reprezentuje udalosť ukončenia morenia a lakovania objednávky pracovníkom C. Pracovník C sa následne priradí prioritne k ďalšej objednávke čakajúcej na montáž kovaní, čím sa naplánuje udalosť **`MoveBetweenWorkplacesEvent`**, alebo v prípade, že žiadna objednávka nečaká na montáž kovaní, k ďalšej objednávke čakajúcej na lakovanie (opäť sa naplánuje udalosť

MoveBetweenWorkplacesEvent). Ak nie je žiadna čakajúca objednávka, pracovník C sa uvoľní. Následne ak je voľný pracovník B, priradí sa k aktuálnej objednávke a na základe jeho polohy sa naplánuje buď udalosť **FinishAssemblyEvent**, alebo niektorá udalosť z **Move...Event**. Ak žiadny pracovník skupiny B nie je dostupný, objednávka sa zaradí do frontu na skladanie. Nakoniec sa aktualizuje GUI.

5.9 Trieda FinishAssemblyEvent

Trieda FinishAssemblyEvent reprezentuje udalosť ukončenia skladania objednávky pracovníkom B. Pracovník B sa následne priradí k ďalšej objednávke čakajúcej na skladanie (ak existuje), čím sa naplánuje udalosť **MoveBetweenWorkplacesEvent**, inak sa stáva voľným. Následne ak je voľný pracovník C a objednávka je typu **CLOSET (skriňa)**, pokúsi sa priradiť voľného pracovníka C a naplánovať udalosť **FinishForgingEvent** (alebo príslušný **Move...Event** podľa polohy pracovníka). Ak C nie je dostupný, objednávka sa zaradí do frontu na montáž kovaní. V prípade iného typu objednávky (TABLE, CHAIR) sa priamo naplánuje udalosť **FinalizationOfOrderEvent**. Nakoniec sa aktualizuje GUI.

5.10 Trieda FinishForgingEvent

Trieda FinishForgingEvent reprezentuje udalosť ukončenia montáže kovaní objednávky pracovníkom C. Pracovník C sa následne priradí prioritne k ďalšej objednávke čakajúcej na montáž kovaní alebo morenie a lakovanie (ak existujú) a následne sa naplánuje udalosť **MoveBetweenWorkplacesEvent**. Ak žiadna takáto objednávka nie je, pracovník sa uvoľní. Bez ohľadu na ďalší postup pracovníka sa zároveň pre aktuálnu objednávku naplánuje udalosť **FinalizationOfOrderEvent**. Nakoniec sa aktualizuje GUI.

5.11 Trieda FinalizationOfOrderEvent

Trieda FinalizationOfOrderEvent reprezentuje udalosť dokončenia celej objednávky. Označí objednávku ako **ukončenú**, uvoľní a vráti montážne miesto do frontu voľných montážnych miest, odstráni objednávku zo zoznamu aktívnych objednávok (sledovaných pre zobrazovanie v GUI) a vypočíta čas jej spracovania, ktorý pridá do replikačnej štatistiky. Nakoniec aktualizuje GUI.

6 Vykonané experimenty

Po úspešnej implementácii simulácie pre stolársku dielnu som vykonal experimenty pre určenie **optimálneho** počtu stolárov jednotlivých skupín tak, aby **priemerný čas dokončenia objednávky** bol menší ako 16h. V tejto časti sa venujem priebehu experimentov a záveru, ktorý som z nich vyvodil.

6.1 Úvodné experimenty

V úvodných experimentoch som najprv vyskúšal kombináciu s minimálnym počtom pracovníkov (1) a následne som zvyšoval počet zamestnancov pre každú skupinu, aby som zistil, ako zvýšenie počtu zamestnancov pri konkrétnej skupine ovplyvní sledovaný výsledok.

Počet replikácií	Počet A	Počet B	Počet C	Výsledok
1000	1	1	1	941,11 h
1000	5	1	1	939,01 h
1000	1	5	1	940,45 h
1000	1	1	5	701,96 h

Po úvodných experimentoch je možné vidieť, že zvýšenie pracovníkov skupín A a B **nemalo výrazný vplyv** výsledok – rozdiel bol iba jedna hodina. Naopak je ale možné vidieť, že zvýšenie pracovníkov C malo **výrazný vplyv** na zníženie priemerného času dokončenia objednávky – **priemerný čas klesol takmer o 240h**.

6.2 Následné experimenty zamerané na skupinu C

Následovnými experimentami som sa snažil zistiť minimálny počet pracovníkov C, pri ktorom by sme dosiahli požadované výsledky (priemerný čas dokončenia objednávky pod 16h). Tento počet som začal zvyšovať od predchádzajúcej kombinácie 1,1,5.

Počet replikácií	Počet A	Počet B	Počet C	Výsledok
1000	1	1	6	642,67 h
1000	1	1	7	583,44 h
1000	1	1	8	524,20 h
1000	1	1	9	464,60 h
1000	1	1	10	409,06 h
1000	1	1	11	396,78 h
1000	1	1	12	396,12 h
1000	1	1	13	395,97 h
1000	1	1	14	396,41 h
1000	1	1	15	396,43 h
1000	1	1	16	396,21 h
1000	1	1	17	396,74 h
1000	1	1	18	396,53 h
1000	1	1	19	395,97 h
1000	1	1	20	396,40h

Po týchto experimentoch je možné vidieť, že postupným zvyšovaním pracovníkov C až po 10 sa **zlepšil výsledok** cca o **60 hodín** pri každom zvýšení. Od počtu pracovníkov skupiny C = 11 sa tento výsledok ale ustálil a ďalšie pridávanie pracovníkov už nespôsobilo nejaké výrazné zlepšenie.

6.3 Následné experimenty zamerané na skupinu A a B

Následovnými experimentami som sa snažil zistiť, aký efekt by malo postupné zvyšovanie pracovníkov A alebo B pri počte pracovníkov C = 11 (počet pri ktorom sa výsledok ustálil pri predchádzajúcich experimentoch).

Počet replikácií	Počet A	Počet B	Počet C	Výsledok
1000	2	1	11	391,47 h
1000	3	1	11	393,43 h
1000	4	1	11	391,58 h
1000	5	1	11	392,00 h
1000	1	2	11	385,69 h
1000	1	3	11	386,36 h
1000	1	4	11	385,99 h
1000	1	5	11	385,63 h
1000	2	2	11	344,92 h

1000	3	3	11	345,26 h
1000	4	4	11	344,06 h
1000	5	5	11	344,07 h

Po týchto experimentoch je možné vidieť, že postupné zvyšovanie iba pracovníkov A nemalo **žiadny výrazný efekt** na výslednú priemernú hodnotu, avšak postupné pridávanie iba pracovníkov B spôsobilo **mierne zlepšenie** (cca o 11 hodín).

Keďže zvyšovanie iba pracovníkov z jednej zo skupín A a B neprinieslo **žiadne výrazné zlepšenia výsledku**, rozhodol som sa preto otestovať, ako by ovplyvnilo postupné pridávanie jedného pracovníka do oboch skupín A a B výslednú priemernú hodnotu.

Tu bolo možné odsledovať **najlepšie zlepšenie spomedzi týchto experimentov** pridaním pracovníka zo skupiny A aj B (cca o 50 hodín). No následným podobným zvyšovaním sa tento priemerný výsledok **opäť ustálil**.

Preto som sa rozhodol pokračovať v experimentoch s kombináciou **2,2,11**.

6.4 Následné experimenty opäť zamerané na skupinu C

Keďže ďalším zvyšovaním skupín A a B od kombinácie **2,2,11** sa výsledok nijak výrazne nezlepšil, rozhodol som sa opäť vyskúšať zvyšovanie pracovníkov C, keďže práve táto skupina mala najvýraznejší efekt na zlepšenie výsledku, kým sa toto zlepšenie nezačalo ustáľovať.

Počet replikácií	Počet A	Počet B	Počet C	Výsledok
1000	2	2	12	285,13 h
1000	2	2	13	225,61 h
1000	2	2	14	167,16 h
1000	2	2	15	109,39 h
1000	2	2	16	53,47 h
1000	2	2	17	19,34 h
1000	2	2	18	13,25 h
1000	1	2	18	385,16 h
1000	1	3	17	384,88 h
1000	2	1	18	391,85 h
1000	3	1	17	391,85 h

Po týchto experimentoch som zaznamenal zvyšovaním pracovníkov C opäť **výrazne zlepšenie výsledného priemeru**, pričom som sa postupným zvyšovaním dostal na požadovaný výsledok pri kombinácii **2,2,18** s priemerným časom dokončenia objednávky **13,25 hodín**. Následne som ešte preskúmal okolie tohto doposiaľ najlepšieho riešenia skúšaním odoberania pracovníkov zo skupín A,B,C tak, aby celkový počet zamestnancov **klesol pod 22** (počet v dop. najlepšom riešení), ale tieto riešenia priniesli **výrazne zhoršenie výsledku** (z cca 13 h na cca 380h).

6.5 Vyčíslenie štatistík pre optimálne riešenie (10 000 replikácií)

Štatistika	Stredná hodnota	95% Interval spoľahlivosti
Priemerný čas dokončenia objednávky	13,23 h	<13,22;13,25>
Priemerný počet nezačatých objednávok	1,707	<1,702;1,713>
Zaťaženie skupiny A	80,38 %	<80,36;80,41>
Zaťaženie skupiny B	80,94 %	<80,92;80,97>
Zaťaženie skupiny C	92,01 %	<91,98;92,04>
Zaťaženie pracovník A1	80,39 %	<80,37;80,42>
Zaťaženie pracovník A2	80,37%	<80,35;80,40>
Zaťaženie pracovník B1	80,95%	<80,92;80,98>
Zaťaženie pracovník B2	80,94%	<80,91;80,97>
Zaťaženie pracovník C1	92,16%	<92,13;92,19>
Zaťaženie pracovník C2	92,15%	<92,12;92,18>
Zaťaženie pracovník C3	92,13%	<92,10;92,16>
Zaťaženie pracovník C4	92,11%	<92,08;92,15>
Zaťaženie pracovník C5	92,10%	<92,07;92,13>
Zaťaženie pracovník C6	92,09%	<92,06;92,12>
Zaťaženie pracovník C7	92,06%	<92,03;92,09>
Zaťaženie pracovník C8	92,05%	<92,02;92,08>
Zaťaženie pracovník C9	92,03%	<92,00;92,06>
Zaťaženie pracovník C10	92,01%	<91,98;92,04>
Zaťaženie pracovník C11	91,98%	<91,95;92,01>
Zaťaženie pracovník C12	91,97%	<91,93;92,00>
Zaťaženie pracovník C13	91,95%	<91,92;91,98>
Zaťaženie pracovník C14	91,93%	<91,90;91,96>
Zaťaženie pracovník C15	91,91%	<91,88;91,94>
Zaťaženie pracovník C16	91,88%	<91,85;91,91>
Zaťaženie pracovník C17	91,86%	<91,83;91,89>
Zaťaženie pracovník C18	91,83%	<91,80;91,86>

7 Záver

V rámci tejto semestrálnej práce sa mi podarilo úspešne implementovať všeobecné udalostné simulačné jadro, na ktorom bola následne postavená simulácia výrobného procesu v stolárskej dielni.

Pomocou simulačných experimentov som testoval rôzne konfigurácie počtu pracovníkov a analyzoval ich vplyv na priemerný čas spracovania objednávky. Cieľom bolo nájsť optimálnu kombináciu a počet pracovníkov, ktorý zabezpečí priemerný čas dokončenia objednávky pod 16 hodín.

Získané dáta a intervaly spoľahlivosti nám umožnili odporučiť optimálne nastavenie systému, ktoré splnilo požiadavku na cieľový čas spracovania pod 16 hodín.